

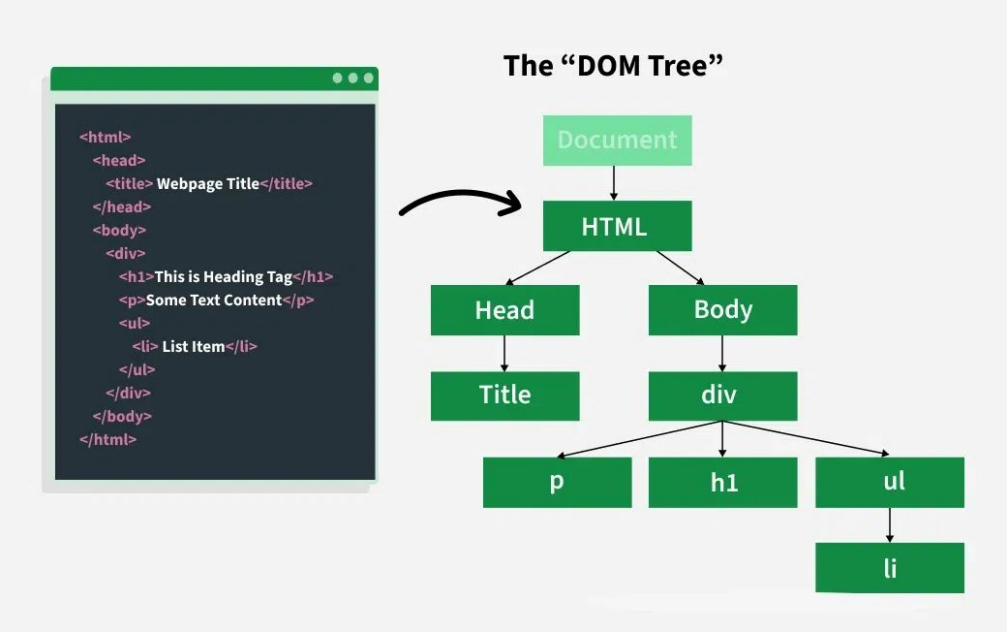
DOM

Document Object Model

HTML DOM

When a web page is loaded, the browser creates a Document Object Model of the page

The HTML DOM model is constructed as a tree of Objects



With the object model, JavaScript gets all the power it needs to create dynamic HTML:

- **add new** HTML elements and attributes
- **change** all the HTML elements, HTML attributes and CSS styles in the page
- **remove** existing HTML elements and attributes
- **create new** HTML events in the page
- **react** to all existing HTML events

HTML DOM

The DOM is a W3C (World Wide Web Consortium) standard for accessing documents

The W3C DOM standard is separated into 3 different parts

Core DOM

- standard model for all document types

XML DOM

- standard model for XML documents

HTML DOM

- standard model for HTML documents



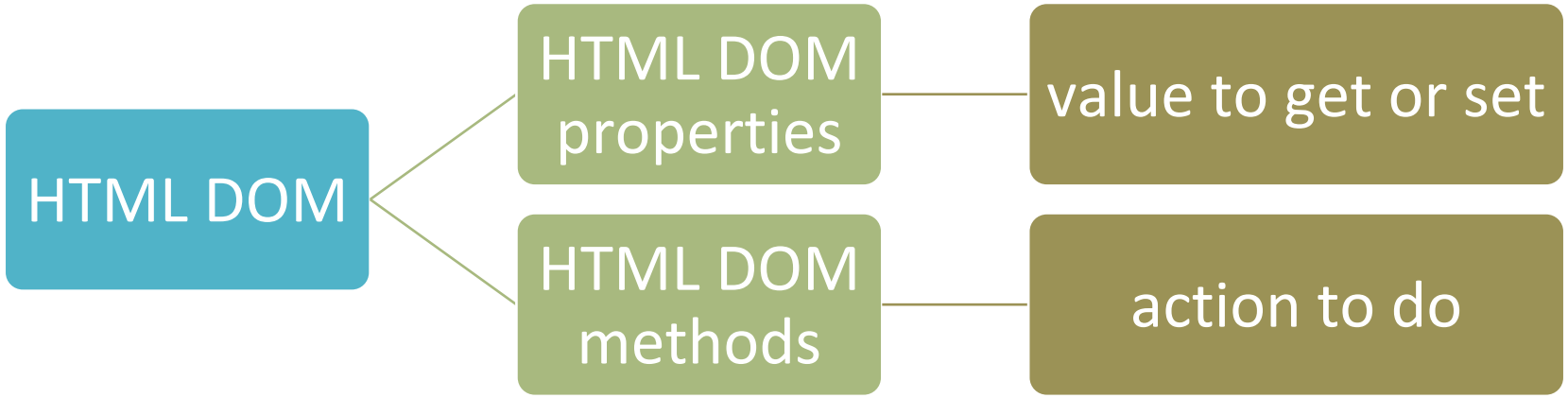
The HTML DOM is a standard for how to get, change, add, or delete **HTML elements**

HTML DOM

In the DOM, all HTML elements are defined as objects.

The programming interface is the properties and methods of each object.

To access elements in HTML page, you always start with accessing the document object




```
document.getElementById("demo").innerHTML = "Hello World!";
```

Finding HTML Objects

The document object represents web page.

to access any element in HTML page by start with accessing the document object.

Method	Description
document.getElementById(*id*)	Find an element by element id
document.getElementsByTagName(*name*)	// object collections Find elements by tag name
document.getElementsByClassName(*name*)	// object collections Find elements by class name
document.querySelector(*name*)	Find first element by specified CSS selector
document.querySelectorAll(*name*)	// object collections Find elements by specified CSS selector

HTML object collections : array-like objects that contain multiple HTML elements

```html
<div id="demo1">
    <div>
        <p class="demo2">text1</p>
        <div>
            <p>text2</p>
            <p>text3</p>
        </div>
    </div>
    <div>
        <div>text4</div>
        <p>text5</p>
    </div>
    <div>
        <p>text6</p>
        <div class="demo2">
            <p>text7</p>
        </div>
    </div>
</div>
```

```html
<script>
    console.log(document.getElementById("demo1").innerText);

    console.log(document.getElementsByTagName("div").length);

    console.log(document.getElementsByTagName("div")[1]
    .getElementsByTagName("p").length);

    console.log(document.getElementsByClassName("demo2").length);

    document.getElementsByTagName("div")[1]
    .getElementsByTagName("p")[1].innerText = "newText1";

    document.getElementsByTagName("div")[3].innerText = "newText2";

    _____________________ // change text7 to newText3

</scri
```

collection example

```
const myCollection = document.getElementsByTagName("p");
for (let i = 0; i < myCollection.length; i++) {
  myCollection[i].style.color = "red";
}
```

```html
<body>
    <div id="demo1">text 1</div>
    <div class="demo">text 2</div>
    <p class="demo">text 3</p>
    <div class="demo">text 4</div>
    <div class="demo">text 5</div>
    <p class="demo">text 6</p>
    <p class="demo">text 7</p>
    <div id="demo2" class="demo">
        <p>text 8</p>
        <div>text 9</div>
        <p>text 10</p>
        <p>text 11</p>
    </div>
<script>
    document.getElementById("demo1").style.backgroundColor = "red";
    document.getElementsByClassName("demo")[2].style.backgroundColor = "blue";
    document.getElementsByTagName("p")[1].style.backgroundColor = "green";
    document.getElementById("demo2").getElementsByTagName("p")[2].style.backgroundColor = "yellow";
    document.getElementById("demo2").querySelectorAll("*")[2].style.backgroundColor = "orange";
    document.querySelectorAll("p:first-child")[0].style.backgroundColor = "brown";
</script>
</body>
```



text 1
text 2
text 3
text 4
text 5
text 6
text 7
text 8
text 9
text 10
text 11

More HTML Objects

property	Description
document.anchors	Deprecated. Do not use it.
document.applets	Deprecated. Do not use it.
document.baseURI	Returns the absolute base URI of the document
document.body	Returns the <body> element
document.cookie	Returns the document's cookie
document.doctype	Returns the document's doctype
document.documentElement	Returns the <html> element
document.documentMode	Returns the mode used by the browser
document.documentURI	Returns the URI of the document
document.domain	Deprecated. Do not use it.
document.domConfig	Deprecated. Do not use it.
document.embeds	Returns all <embed> elements
document.forms	Returns all <form> elements

property	Description
document.head	Returns the <head> element
document.images	Returns all <img> elements
document.implementation	Returns the DOM implementation
document.inputEncoding	Returns the document's encoding (character set)
document.lastModified	Returns the date and time the document was updated
document.links	Returns all <area> and <a> elements that have a href attribute
document.readyState	Returns the (loading) status of the document
document.referrer	Returns the URI of the referrer (the linking document)
document.scripts	Returns all <script> elements
document.strictErrorChecking	Returns if error checking is enforced
document.title	Returns the <title> element
document.URL	Returns the complete URL of the document

Changing HTML Elements

Property	Description
element.innerHTML = *new html content*	Change the inner HTML of an element
element.attribute = *new value*	Change the attribute value of an HTML element
element.style.*property* = *new style*	Change the style of an HTML element
Method	**Description**
element.setAttribute*(attribute, value)*	Change the attribute value of an HTML element
document.write(value)	write directly to the HTML output stream

Changing HTML Elements

Changing HTML Content

```
document.getElementById(id).innerHTML = new HTML
```

```
document.getElementById("demo1").innerHTML = "<b>New text!</b>";
```

```
const element = document.getElementById("demo1");
element.innerHTML = "<b>New text!</b>";
```

Changing the Value of an Attribute

```
document.getElementById(id).attribute = new value
```

```
document.getElementById("myImage").src = "landscape.jpg";
```

document.write()

```
document.write(value)
```

```
<p>welcome <script>document.write("John Doe");</script></p>
```

Changing HTML Style

document.getElementById(id).style.property = new style

```
document.getElementById("p2").style.color = "blue";
```

```
<p id="p1">This is a text.</p>

<input type="button" value="Hide text"
onclick="document.getElementById('p1').style.visibility='hidden'">

<input type="button" value="Show text"
onclick="document.getElementById('p1').style.visibility='visible'">
```

DOM Event

DOM events are signals exposed by the browser that you can use to run a piece of JavaScript code

These DOM events occur when the user interacts with the application we've created, such as clicking a button or typing letters into an input field.

HTML DOM allows you to assign events to HTML elements using JavaScript

```
addEventListener(event, function)
```

```
<button id="btn1">click</button>
<button id="btn2">click</button>
<script>
    let alertText=()=>{window.alert("Hello");}
    document.getElementById("btn1").addEventListener('click',alertText);
    document.getElementById("btn2").onclick = alertText;
</script>
```

the most common events

Event	Event is fired
click	When you press down and release the primary mouse button. Used to track buttons and clickable elemennts
mousemove	When you move the mouse cursor
mouseover	When you move the mouse cursor over an element. It's like the CSS hover state
mouseout	When your mouse cursor moves outside the boundaries of an element
dblclick	When you click twice
DOMContentLoaded	When the DOM content is fully loaded
keydown	When you press a key on your keyboard
keyup	When you release a key on your keyboard
submit	When a form is submitted

full list of DOM event types

https://en.wikipedia.org/wiki/DOM_event

Note

Type	Attribute
click	onclick
dblclick	ondblclick
mousedown	onmousedown
mouseup	onmouseup

information contained inside the event object

```
let alertText = (event) => {console.log(event);}
```



```
<button id="btn">click</button>
<script>
    let alertText = () => { alert("hello") }
    document.getElementById("btn").addEventListener('click', alertText);
    document.getElementById("btn").ondblclick = alertText;

    document.addEventListener('keydown', event => {
        console.log(`A key is pressed: ${event.key}`);
    });
    document.addEventListener('keyup', event => {
        console.log(`A key is released: ${event.key}`);
    });
    document.addEventListener('mousedown', event => {
        console.log(`The mouse is pressed`);
    });
    document.addEventListener('mouseup', event => {
        console.log(`The mouse is released`);
    });
</script>
```

```
A key is pressed: a
A key is released: a
```

```
The mouse is pressed
The mouse is released
```

Note: The keyboard and mouse events are usually attached to the document object

addEventListener() method also allows you to attach multiple listeners to the same element

DOM Event – Remove EventListener

Remove Event Listeners

element.removeEventListener(event, function);

```
button.removeEventListener('click', alertText);
```

element.eventattribute = null

```
myBtn.onclick = null;
```

If you pass a nameless function to the addEventListener() method, then that event can't be removed

```
button.addEventListener('click', function (event) {
    alert('Button save is clicked');
});
```



This event can't be removed

The onload and onunload Events

The `onload` and `onunload` events are triggered when the user enters or leaves the page.

```
<body onload="checkCookies()">
```

The oninput Event

The `oninput` event is often to some action while the user input data.

```
<input type="text" id="fname" oninput="upperCase()">
```

The onchange Event

The `onchange` event is often used in combination with validation of input fields.

```
<input type="text" id="fname" onchange="upperCase()">
```

The onmouseover and onmouseout Events

The `onmouseover` and `onmouseout` events can be used to trigger a function when the user mouses over, or out of, an HTML element

Forms - Data Validation

Data validation is the process of ensuring that user input is clean, correct, and useful

Validation can be defined by many different methods, and deployed in many different ways.

Server side validation

performed by a web server, after input has been sent to the server.

Client side validation

performed by a web browser, before input is sent to a web server.

Forms - HTML Constraint Validation

HTML constraint validation is based on

HTML Input Attributes

CSS Pseudo Selectors

DOM Properties and Methods



javaScript

Forms - HTML Constraint Validation

Constraint Validation HTML Input Attributes

Attribute	Description
disabled	Specifies that the input element should be disabled
max	Specifies the maximum value of an input element
min	Specifies the minimum value of an input element
pattern	Specifies the value pattern of an input element
required	Specifies that the input field requires an element
type	Specifies the type of an input element

Forms - HTML Constraint Validation

Constraint Validation CSS Pseudo Selectors

Selector	Description
:disabled	Selects input elements with the "disabled" attribute specified
:invalid	Selects input elements with invalid values
:optional	Selects input elements with no "required" attribute specified
:required	Selects input elements with the "required" attribute specified
:valid	Selects input elements with valid values

Forms - HTML Constraint Validation

Constraint Validation DOM Properties and Methods

```
<form name="myForm" action="/action_page.php" onsubmit="return validateForm()" method="post">
Name: <input type="text" name="fname">
<input type="submit" value="Submit">
</form>
```

```
function validateForm() {
  let x = document.forms["myForm"]["fname"].value;
  if (x == "") {
    alert("Name must be filled out");
    return false;
  }
}
```



no explicit return statement is return
undefined
undefined is not evaluated as a false value

Name: number:
Submit

```
<form name="myForm" action="/action_page.php" onsubmit="return validateForm()" method="post">
    Name: <input type="text" id="fname" name="fname">
    number:<input id="numb">
    <div id="errText"></div>
    <input type="submit" value="Submit">
</form>
```

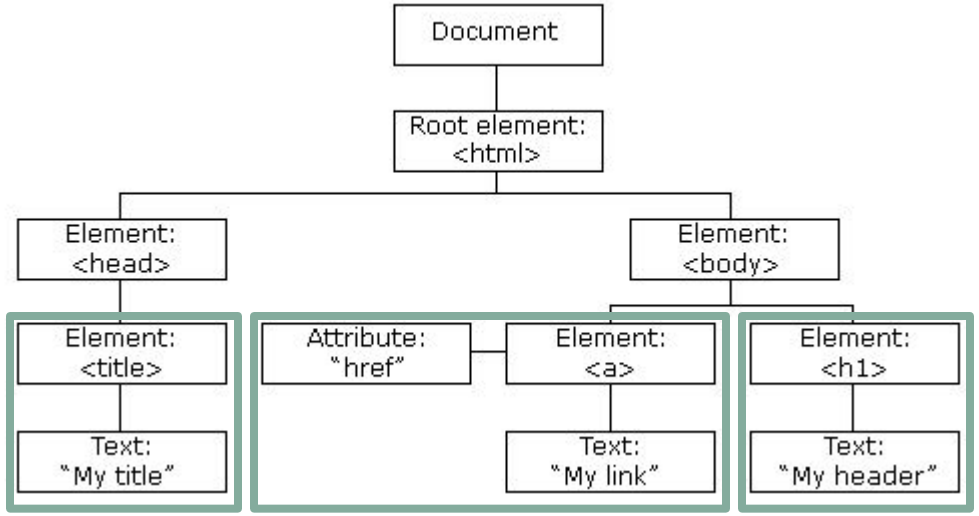
```
function validateForm() {
    let errList=[];
    let x = document.forms["myForm"]["fname"].value;
    if (x == "") { errList.push("Name must be filled out");
    }
    let n = document.forms["myForm"]["numb"].value;
    if (isNaN(n) || n < 1 || n > 10) { errList.push("number not valid");
    }
    if(errList.length > 0)
    {   document.getElementById("errText").innerHTML = errList;
        errList=[];
        return false;
```

DOM Nodes

Everything in an HTML document is a node

With the HTML DOM, all nodes in the node tree can be accessed by JavaScript.

New nodes can be created, and all nodes can be modified or deleted



Document node - The entire document
Element node - Every HTML element
text nodes - text inside HTML elements
attribute node - Every HTML attribute is an
comment nodes - comments

DOM Nodes

Node Relationships

The nodes in the node tree have a hierarchical relationship to each other

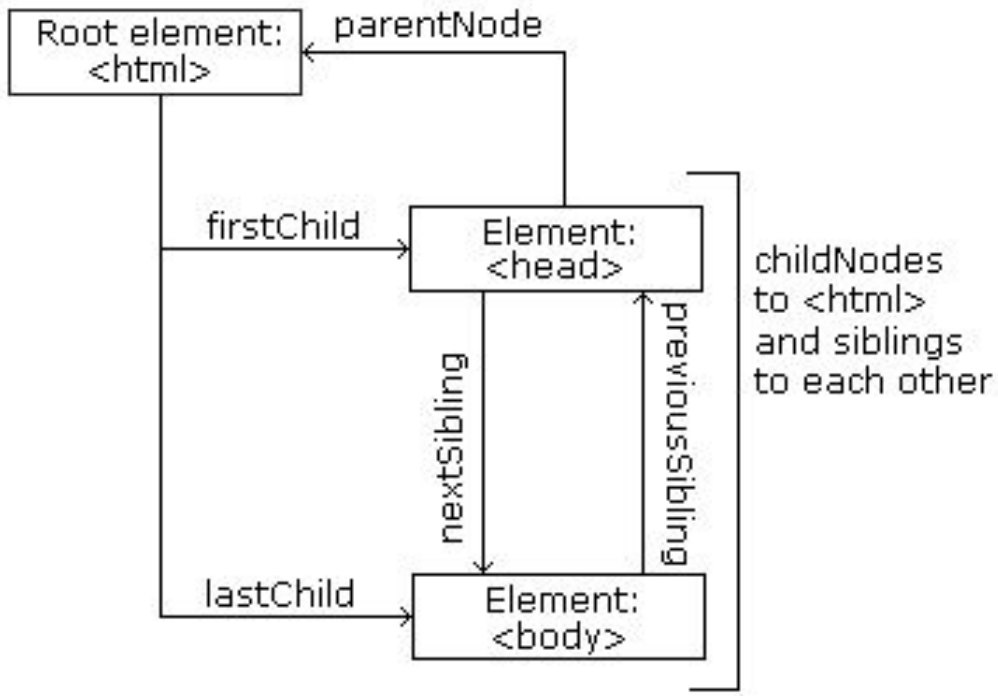
In a node tree, the top node is called the root

```
<html>
  <head>
    <title>DOM Tutorial</title>
  </head>

  <body>
    <h1>DOM Lesson one</h1>
    <p>Hello world!</p>
  </body>

</html>
```

<h1> and <p> are siblings

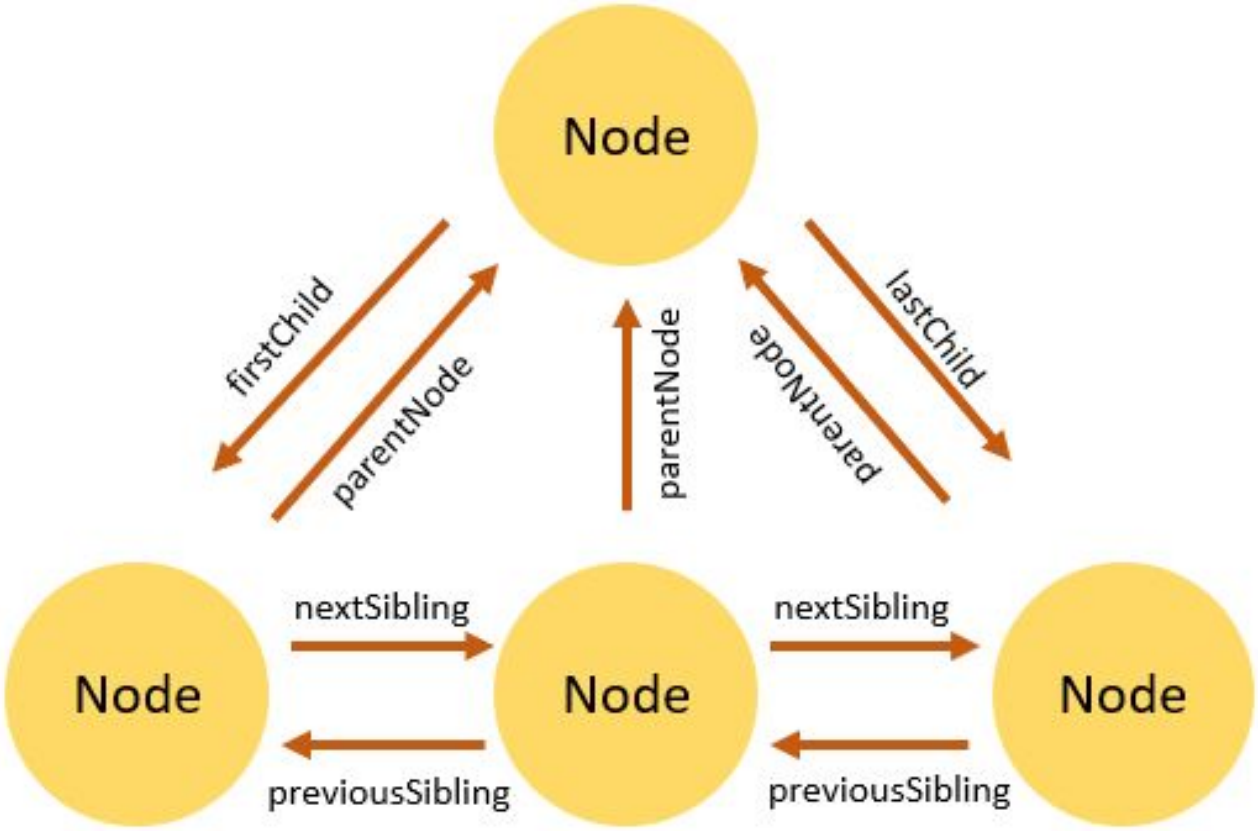


DOM Nodes

Navigating Between Nodes

use the node properties to navigate between nodes with JavaScript

- parentNode
- childNodes[nodenumber]
- firstChild
- lastChild
- nextSibling
- previousSibling



Child Nodes and Node Values

The element node contains a text node with the text value.

```
<p id="demo">Hello</p>
```

the innerHTML property access the value of the text node

```
document.getElementById("demo").innerHTML;
```

Hello

```
document.getElementById("demo").firstChild.nodeValue;
```

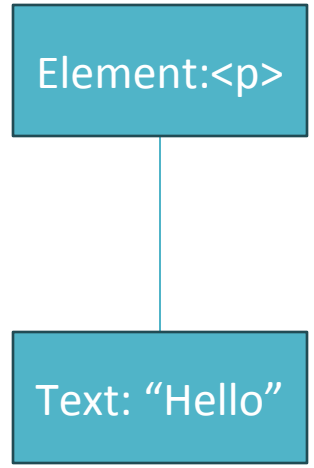
Hello

```
document.getElementById("demo").childNodes[0].nodeValue;
```

Hello

```
document.getElementById("demo").nodeValue
```

→ null



To access to the full document

```
console.log(document.documentElement.innerHTML);
```



```
<head>
  <title>Document</title>
</head>
<body>
  <div>
    <p>Hello</p>
    <p>World</p>
  </div>
  <script>
    console.log(document.documentElement.innerHTML);
  </script>
</body>
```

```
console.log(document.body.innerHTML);
```



```
<div>
    <p>Hello</p>
    <p>World</p>
</div>

<script>
    console.log(document.body.innerHTML);
</script>
```

The nodeType Property

```
<h1 id="id01">My First Page</h1>
```

```
document.getElementById("id01").nodeType
```

1

```
document.getElementById("id01").firstChild.nodeType
```

3

Node	Type	Example
ELEMENT_NODE	1	<h1 class="heading">W3Schools</h1>
ATTRIBUTE_NODE	2	class = "heading" (deprecated)
TEXT_NODE	3	W3Schools
COMMENT_NODE	8	<!-- This is a comment -->
DOCUMENT_NODE	9	The HTML document itself (the parent of <html>)
DOCUMENT_TYPE_NODE	10	<!Doctype html>

DOM Nodes

Adding and Deleting Elements

Method	Description
document.createElement(*element*)	Create an HTML element
document.removeChild(*element*)	Remove an HTML element
document.appendChild(*element*)	Add an HTML element
document.replaceChild(*new, old*)	Replace an HTML element
document.write(*text*)	Write into the HTML output stream

Creating New HTML Elements (Nodes)



createAttribute()
createComment()
createDocumentFragment()
createElement()
createEvent()
createTextNode()

```
<div id="div1">
    <p id="p1">paragraph1</p>
    <p id="p2">paragraph2</p>
</div>

<script>
    const para = document.createElement("p");
    const node = document.createTextNode("new paragraph");
    para.appendChild(node);

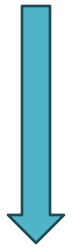
    const element = document.getElementById("div1");
    element.appendChild(para);
</script>
```

paragraph1

paragraph2



new paragraph

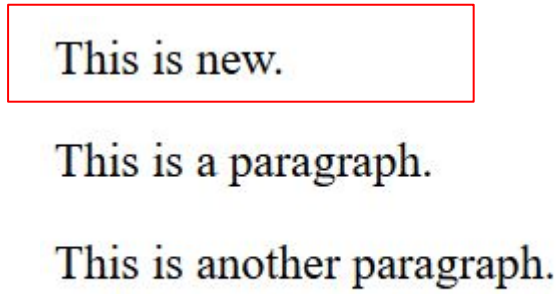


```
para.innerText = "new paragraph";
```

insertBefore()

```
<div id="div1">
  <p id="p1">This is a paragraph.</p>
  <p id="p2">This is another paragraph.</p>
</div>
<script>
const para = document.createElement("p");
const node = document.createTextNode("This is new.");
para.appendChild(node);

//const parent = document.getElementById("p1").parentNode;
const parent = document.getElementById("div1");
const child = document.getElementById("p1");
parent.insertBefore(para, child);
</script>
```




```
// Insert the new element before the first child
parent.insertBefore(newElement, parent.firstChild);
```

```
// insert after child
parent.insertBefore(newElement, child.nextSibling);
```

Note: There is no insertAfter() method

remove()

```
<div id="d1">
  <p id="p1">This is a paragraph.</p>
  <p id="p2">This is another paragraph.</p>
</div>

<script>
const elmnt = document.getElementById("p1");
elmnt.remove();
</script>
```



This is another paragraph.

removeChild()

```
<script>
  const parent = document.getElementById("d1");
  const child = document.getElementById("p1");
  parent.removeChild(child);
  //child.parentNode.removeChild(node);
</script>
```

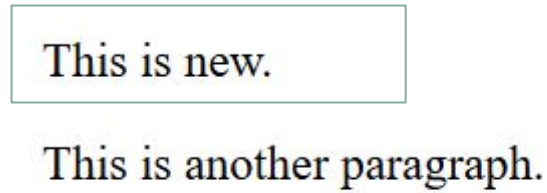
```
// remove firstchild
parent.removeChild(parent.firstChild)
;
```

replaceChild()

```
<div id="div1">
  <p id="p1">This is a paragraph.</p>
  <p id="p2">This is another paragraph.</p>
</div>

<script>
const para = document.createElement("p");
const node = document.createTextNode("This is new.");
para.appendChild(node);

const parent = document.getElementById("div1");
const child = document.getElementById("p1");
parent.replaceChild(para, child);
</script>
```



`insertBefore()` method inserts a child node before an existing child

`insertAdjacentElement()` method inserts a an element into a specified position

`insertAdjacentHTML()` method inserts HTML code into a specified position.

`insertAdjacentText()` method inserts a a text into a specified position.

Legal positions:

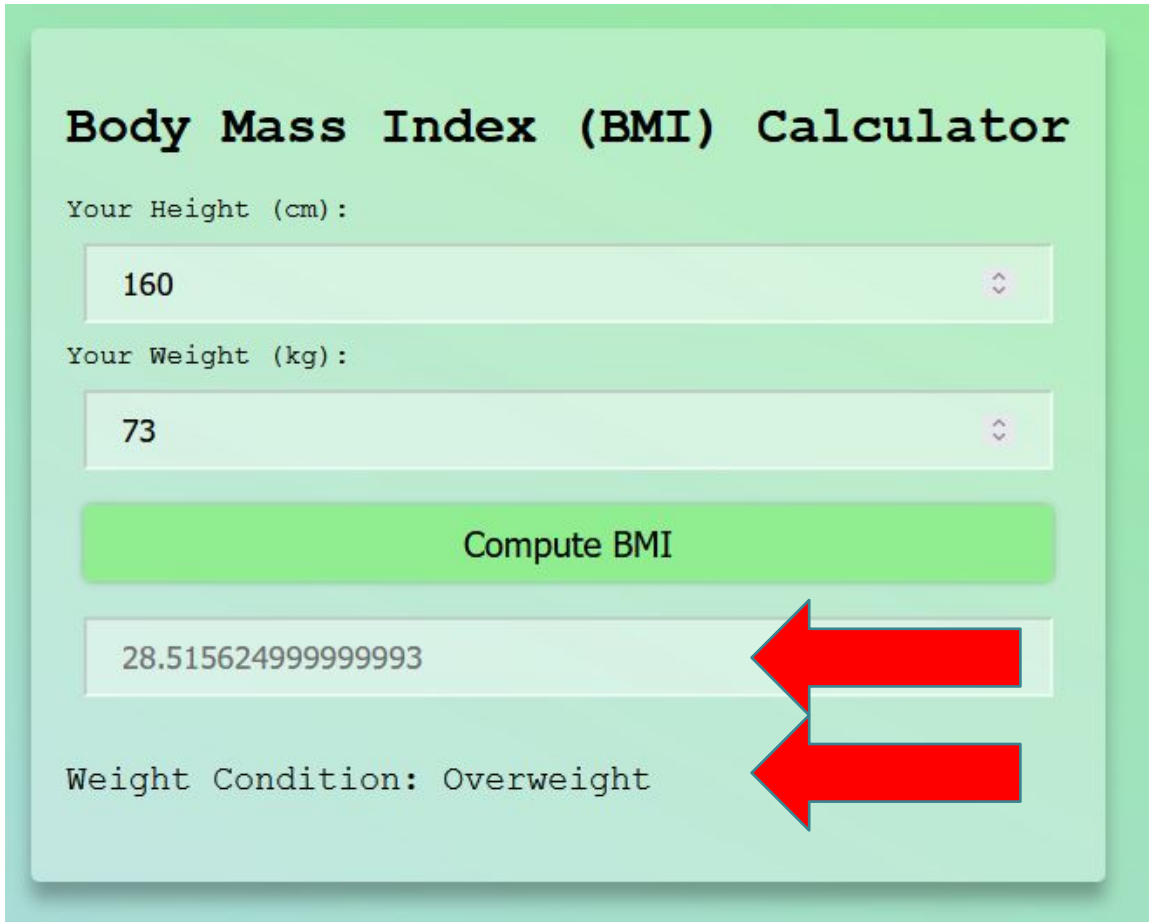
Value	Description
afterbegin	After the beginning of the element (first child)
afterend	After the element
beforebegin	Before the element
beforeend	Before the end of the element (last child)

```
<!-- beforebegin -->
<p>
<!-- afterbegin -->
foo
<!-- beforeend -->
</p>
<!-- afterend -->
```



Todo

1. Body Mass Index (BMI) Calculator



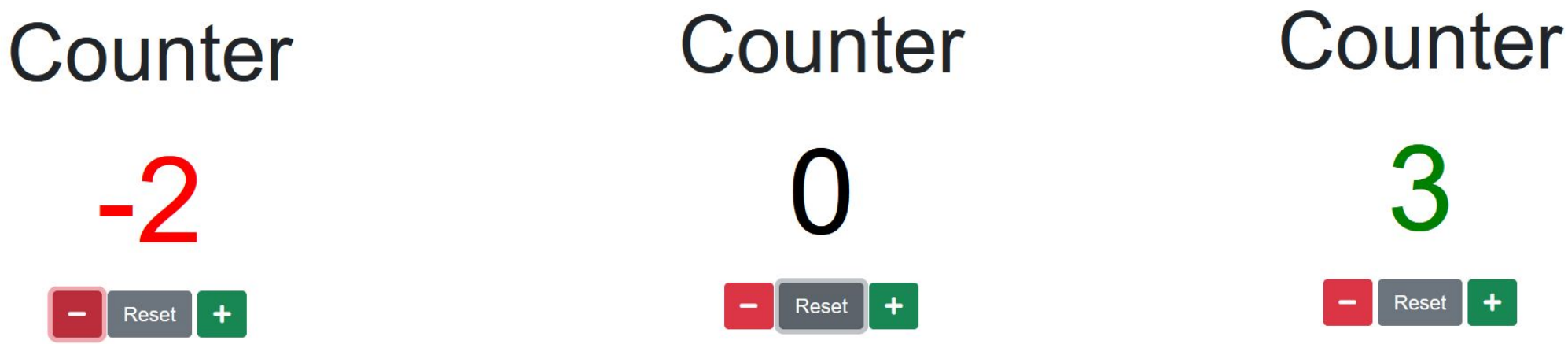
Compute BMI button :
calculate the BMI and weight Condition

$$BMI = \frac{\text{Weight in kilogram}}{\left(\text{Height in meter}\right)^2}$$

Underweight	Normal weight	Overweight	Obese
< 18.5	18.5 - 25	25 - 30	> 30

2 Counter

Create increase decrease and reset button
The color of number change by the counter value



3 calculator



4 Todo list

Add task to the list

